Data types:

undefined null true false object array bigint

dedicated values:
NAN INFINITY HOLE

dedicated objects:
global Symbol

Language features:

- object / class inheritance (e.g., reference to super())
- module / namespace
- lexical environment (closure)
- getter / setter accessors
- dynamic typing (e.g., instanceof and in operator)
- for .. in .. iterator
- regular expressions
- Async / generator/ await
- acc could be a list (e.g., spread syntax)
- sendable classes
- # private fields
- patch mode
- lazy import

可以翻译的指令比例 = 50 / 105 = 47.6%

Bytecode inst	Example ArkTS source code	LLVM IR
ldundefined	acc = undefined	✔
ldnull	acc = null	✔
ldtrue	acc = true	✔
ldfalse	acc = false	✔
createemptyobject	let obj = {};	✔
createemptyarray RR	let arr = [];	✔
createarraywithbuffer RR, @AAAA	let arr = [1, 2, 3];	✔
	let obj = { x: 1, y: 2 };	✔

createobjectwithbuffer RR, @AAAA		

newobjrange RR, +AA, vBB	class Person {
	  constructor(name: string, age: number) {}
	}
	
	// vBB holds the class, vBB+1 holds "Alice", vBB+2 holds 30
	
	let p = new Person("Alice", 30);
		✔

newlexenv +AA, wide.newlexenv +AAAA	NA	NA
add2 RR, vAA	acc = acc + vAA	✔
sub2 RR, vAA	acc = vAA - acc	✔
mul2 RR, vAA	acc = vAA * acc	✔
div2 RR, vAA	acc = vAA / acc	✔
mod2 RR, vAA	acc = vAA % acc	✔
eq RR, vAA	acc = vAA == acc	✔
noteq RR, vAA	acc = vAA != acc	✔
less RR, vAA	acc = vAA < acc	✔
lesseq RR, vAA	acc = vAA <= acc	✔
greater RR, vAA	acc = vAA > acc	✔
greatereq RR, vAA	acc = vAA >= acc	✔
shl2 RR, vAA	acc = vAA << acc	✔
shr2 RR, vAA	acc = vAA >>> acc	✔
ashr2 RR, vAA	acc = vAA >> acc	✔
and2 RR, vAA	acc = vAA & acc	✔
or2 RR, vAA	acc = vAA | acc	✔
xor2 RR, vAA	acc = vAA ^ acc	✔
exp RR, vAA	acc = vAA ** acc	✔

typeof RR	acc = typeof acc	❌
tonumber RR	acc = ToNumber (acc)	STUB
tonumeric RR	acc = ToNumeric (acc)	STUB
neg RR	acc = -acc	✔
not RR	acc = ~acc	✔
inc RR	acc = acc + 1	✔
dec RR	acc = acc - 1	✔
istrue callruntime.istrue +RR	acc = acc == true	✔
isfalse callruntime.isfalse +RR	acc = acc == false	✔

isin RR, vAA	const car = { make: 'Toyota' };
	
	console.log('make' in car);
	// true
	
	console.log('toString' in car);
	// true (inherited from Object.prototype)
		❌

instanceof RR, vAA	const numbers = [1, 2, 3];
	
	console.log(numbers instanceof Array); // true
	
	console.log(numbers instanceof Object); // true (Array inherits from Object)
		❌

strictnoteq RR, vAA	acc = acc !== vAA	❌
stricteq RR, vAA	acc = acc === vAA	❌
callarg0 RR	acc = call acc()	✔
callarg1 RR, vAA	acc = call acc(vAA)	✔
callargs2 RR, vAA, vBB	acc = call acc(vAA, vBB)	✔
callargs3 RR, vAA, vBB, vCC	acc = call acc(vAA, vBB, vCC)	✔
callthis0 RR, vAA	class Greeter { greet(): void {	✔

	console.log("Hello");
	}
	}
	
	// acc <- greet, vAA <- g
	let g = new Greeter();
	g.greet();	

callthis1 RR, vAA, vBB	class Calculator {
	addOne(n: number): number {
	return n + 1;
	}
	}
	
	// acc <- addOne, vAA <-calc, vBB <-5
	let calc = new Calculator();
	calc.addOne(5);
		✔

callthis2 RR, vAA, vBB, vCC	acc = vAA -> acc(vBB, vCC)	✔
callthis3 RR, vAA, vBB, vCC, vDD	acc = vAA -> acc(vBB, vCC,vDD)	✔
callthisrange RR, +AA, vBB wide.callthisrange +AAAA, vBB	acc = vBB -> acc(vBB+1, …, vBB+A)	✔

supercallthisrange RR, +AA, vBB
 	
wide.supercallthisrange +AAAA, vBB
	acc = super(vBB, …, vBB + A - 1)
	
	class Base {
	constructor(x: number, y: number) {}
	}
	
	class Derived extends Base {
	constructor() {
	super(10, 20);
	}
	}
		❌

definefunc RR, @AAAA, +BB	acc = create_func_obj_for_method(A)	✔

definemethod RR, @AAAA, +BB	obj = create_func_obj_for_method(A);
	
	// For super() calls obj.HomeObject = acc;
	
	acc = obj;
		❌

defineclasswithbuffer RR, @AAAA, @BBBB, +CCCC, vDD	class Animal {}
	
	class Cat extends Animal {
	  meow() {}
	}
	
	// @AAAA: Points to the constructor of Cat.
	@BBBB: Points to a literal array buffer containing keys and values for methods like meow.
	vDD: Holds the reference to the parent class Animal.
		✔

getnextpropname vAA	const data = { a: 1, b: 2, c: 3 };
	
	for (const key in data) {
	  console.log(key);
	}
	
	// This instruction is the core mechanism of a for...in loop.
	
	When the loop starts, the runtime creates an internal iterator for the keys of data.
	In every iteration of the loop, getnextpropname is called on that iterator (stored in register vAA) to fetch the next key string (e.g., "a", then "b").
	The result is stored in acc and subsequently
		❌

	assigned to the variable key.	
ldobjbyvalue RR, vAA	acc = vAA.acc	✔
stobjbyvalue RR, vAA, vBB	vAA.vBB = acc	✔

ldsuperbyvalue RR, vAA	acc = super.acc
	
	class Base {
	  data = "Base Data";
	  getData() { return this.data; }
	}
	
	class Derived extends Base {
	  test(key: string) {
	  // The operation below compiles to ldsuperbyvalue
	  return super[key];
	  }
	}
	
	let d = new Derived();
	d.test("getData"); // Accesses super["getData"]
		❌

ldobjbyindex RR, +AAAA wide.ldobjbyindex +AAAAAAAA	acc = acc.A	✔
stobjbyindex RR, vAA, +BBBB wide.stobjbyindex vAA, +BBBBBBBB	vAA.B = acc	✔
ldlexvar +A, +B wide.ldlexvar +AAAA, +BBBB	NA	✔
stlexvar +A, +B wide.stlexvar +AAAA, +BBBB	NA	✔
lda.str @AAAA	acc = “hi”	✔
tryldglobalbyname RR,	acc = global A	✔

@AAAA		
trystglobalbyname RR, @AAAA	global A = acc	✔
ldglobalvar RRRR, @AAAA	acc = global A	✔
stglobalvar RRRR, @AAAA	global A = acc	✔
ldobjbyname RR, @AAAA	acc = acc.A	✔
stobjbyname RR, @AAAA, vBB	vBB.A = acc	✔
mov vA, vB	vA = vB	✔
mov vAA, vBB	vAA = vBB	✔
ldsuperbyname RR, @AAAA	acc = super.A	❌
stconsttoglobalrecord RRRR, @AAAA	// Defined at the top level (global scope) const PI = 3.14159;	✔

sttoglobalrecord RRRR, @AAAA	// Defined at the top level (global scope)
	let count = 0;
	
	// Later in the global scope
	count = 1;
		✔

ldthisbyname RR, @AAAA	acc = this.A	✔
stthisbyname RR, @AAAA	this.A = acc	✔
ldthisbyvalue RR	acc = this.acc	✔
stthisbyvalue RR, vAA	this.vAA = acc	✔
jmp +AA,jmp +AAAA,jeqz +AA,jeqz +AAAA,jnez +AA,jstricteqz +AA,jnstricteqz +AA,jeqnull +AA,jnenull +AA,jstricteqnull		✔

+AA,jnstricteqnull +AA,jequndefined +AA,jneundefined +AA,jstrictequndefined +AA,jnstrictequndefined +AA,jeq vAA, +BB,jne vAA, +BB,jstricteq vAA, +BB,jnstricteq vAA, +BB,		
lda vAA	acc = vAA	✔
sta vAA	vAA = acc	✔
ldai +AAAAAAAA fldai +AAAAAAAAAAAAAAAA	acc = int / float literal	✔
return returnundefined		✔
getpropiterator		❌
getiterator RR		❌
closeiterator RR, vAA		❌
poplexenv	NA	✔
ldnan		✔
ldinfinity		✔
getunmappedargs	Stores the arguments of the current function in the acc.	✔
ldglobal	Stores the global object in the acc.	✔
ldnewtarget	deprecated	
ldthis	acc = this	✔
ldhole	acc = hole	✔

createregexpwithliteral RR, @AAAA, +BB
createregexpwithliteral RRRR, @AAAA, +BB
	// Case 1: No flags (B = 0)
	let re1 = /abc/;
	
	// Case 2: With flags (B = 3, which is 1 | 2)
	let re2 = /test/gi;
	
	// Case 3: With different
		❌

	flags (B = 4) let re3 = /foo/m;	
callrange RR, +AA, vBB wide.callrange +AAAA, vBB	acc = acc(vBB, … ,vBB + A - 1)	✔
gettemplateobject RR	acc =GetTemplateObject(acc)	❌
setobjectwithproto RR, vAA	acc.__proto__ = vAA	✔
stownbyvalue RR, vAA, vBB	vAA.vBB = acc	✔
stownbyindex RR, vAA, +BBBB wide.stownbyindex vAA, +BBBBBBBB	vAA.B = acc	✔
stownbyname RR, @AAAA, vBB	vBB.A = acc	✔
getmodulenamespace +AA wide.getmodulenamespace +AAAA		❌
stmodulevar +AA wide.stmodulevar +AAAA		❌
ldlocalmodulevar +AA wide.ldlocalmodulevar +AAAA		❌
ldexternalmodulevar +AA wide.ldexternalmodulevar +AAAA		❌
stownbynamewithnameset RR, @AAAA, vBB	Stores a function in an object property and simultaneously sets the function's .name property to match the property key. This implements ES6 function name inference.	✔

stownbyvaluewithnameset RR, vAA, vBB		✔
asyncgeneratorreject vAA	acc = AsyncGeneratorReject (vAA, acc)	❌
ldsymbol	acc = Symbol	✔
asyncfunctionenter	acc = create_async_func_obj()	❌
ldfunction	acc = current func obj	❌
debugger	NA	NA
creategeneratorobj vAA		❌
createiterresultobj vAA, vBB		❌
createobjectwithexcludedkeys +AA, vBB, vCC		❌

newobjapply RR, vAA	class Point {
	  x: number;
	  y: number;
	  constructor(x: number, y: number) {
	  this.x = x;
	  this.y = y;
	  }
	}
	
	function createPoint(args: [number, number]) {
	  // The spread operator (...) here triggers the generation of newobjapply
		✔

	// 'args' is loaded into the accumulator (acc), and 'Point' is the class object (vAA).
	return new Point(...args);
	}	

newlexenvwithname +AA, @BBBB wide.newlexenvwithname +AAAA, @BBBB		✔
createasyncgeneratorobj vAA		❌
asyncgeneratorresolve vAA, vBB, vCC		❌

supercallspread RR, vAA	class Base {
	constructor(a: number, b: number) {}
	}
	
	class Derived extends Base {
	constructor(numbers: number[]) {
	// The spread operator (...) inside super() triggers supercallspread
	// 'numbers' is the parameter list (vAA), and the class object is in the accumulator.
	super(...numbers);
	}
	}
		✔

apply RR, vAA, vBB		✔

supercallarrowrange RR, +AA, vBB wide.supercallarrowrange +AAAA, vBB		✔

definegettersetterbyvalue vAA, vBB, vCC, vDD	function createDynamicObject(propName: string) {
	  return {
	  // Using a computed property name [propName] for accessors triggers definegettersetterbyvalue.
	  // vAA: The object being created.
	  // vBB: The variable 'propName'.
	  // vCC: The getter function.
	  // vDD: The setter function.
	  get [propName]() {
	  return "some value";
	  },
	  set [propName](val: string) {
	  console.log(val);
	  }
	  };
	}
		❌

dynamicimport		❌
resumegenerator		❌
getresumemode		❌
gettemplateobject RRRR		❌

delobjprop vAA	del vAA.acc	✔
suspendgenerator vAA		❌
asyncfunctionawaituncaught vAA		❌
copydataproperties vAA	Copies all properties of the object stored in the acc to A and stores A in the acc.	❌

starrayspread vAA, vBB	function buildArray(middleItems: number[]) {
	  // The spread operator (...) used inside the array literal [...] triggers starrayspread.
	  // vAA: The new array being constructed.
	  // acc: The 'middleItems' array being spread.
	  // vBB: The index where the spread begins (index 1 in this example).
	  const result = [0, ...middleItems, 4];
	  return result;
	}
		❌

stsuperbyvalue RR, vAA, vBB	super.vBB = vAA	✔
asyncfunctionresolve vAA		❌
asyncfunctionreject vAA		❌

copyrestargs +AA wide.copyrestargs +AAAA		✔
stsuperbyname RR, @AAAA, vBB	super.A = acc	✔
ldbigint @AAAA		❌
setgeneratorstate		❌
getasynciterator		❌

ldprivateproperty RR, +AAAA, +BBBB	class Wallet {
	  // The # syntax denotes a hard-private field.
	  #balance: number;
	
	  constructor(initial: number) {
	  // Writing to a private field triggers stprivateproperty.
	  // vCC: The object instance ('this').
	  // acc: The value being assigned ('initial').
	  // The key for '#balance' is retrieved from the lexical environment (slot B, level A).
	  this.#balance = initial;
	  }
	
	  checkBalance() {
	  // Reading a private field triggers ldprivateproperty.
	  // acc: The object instance ('this').
	  // The key for
		✔

	'#balance' is retrieved from the lexical environment to lookup the value.
	  return this.#balance;
	  }
	}	

stprivateproperty RR, +AAAA, +BBBB, vCC		✔
testin RR, +AAAA, +BBBB		❌
definefieldbyname RR, @AAAA, vBB	vBB.A = acc	✔
definepropertybyname RR, @AAAA, vBB	vBB.A = acc	✔
callruntime.notifyconcurrentresult		❌
wide.createobjectwithexcludedkeys +AAAA, vBB, vCC		❌
throw throw.patternnoncoercible throw.notexists throw.deletesuperproperty throw.constassignment vAA throw.ifnotobject vAA		✔

throw.undefinedifhole vAA, vBB throw.ifsupernotcorrectcall +AA throw.undefinedifholewithname @AAAA		
callruntime.definefieldbyvalue RR, vAA, vBB	vBB.vAA = acc	✔
wide.newobjrange +AAAA, vBB		✔
callruntime.definefieldbyindex RR, +AAAAAAAA, vBB	vBB.vAA = acc	✔

callruntime.topropertykey	const complexKey = {
	  toString() {
	  return "dynamic_prop";
	  }
	};
	
	const obj = {};
	
	// The variable 'complexKey' is in the accumulator.
	// Before the assignment can happen, the runtime must convert the object
	// into a valid property key (the string "dynamic_prop").
	// This conversion logic is handled by callruntime.topropertykey.
	obj[complexKey] = 100;
		❌

callruntime.createprivat		✔

eproperty +AAAA, @BBBB		
callruntime.defineprivateproperty RR, +AAAA, +BBBB, vCC		✔
callruntime.callinit +RR, vAA	ldobjbyindex->acc()	✔
callruntime.definesendableclass RRRR, @AAAA, @BBBB, +CCCC, vDD		❌
callruntime.ldsendableclass +AAAA		❌
callruntime.ldsendableexternalmodulevar +AA		❌
callruntime.wideldsendableexternalmodulevar +AAAA		❌
callruntime.newsendableenv +AA callruntime.widenewsendableenv +AAAA		❌
callruntime.stsendablevar +A +B		❌

callruntime.widestsendablevar +AAAA +BBBB		
callruntime.ldsendablevar +A +B callruntime.wideldsendablevar +AAAA +BBBB		❌
wide.ldpatchvar +AAAA		❌
wide.stpatchvar +AAAA		❌
callruntime.ldlazymodulevar +AA callruntime.wideldlazymodulevar +AAAA callruntime.ldlazysendablemodulevar +AA callruntime.wideldlazysendablemodulevar +AAAA		❌